

F1R3COMB

A compiler from kernel rho to the name-free rho combinators,
and a machine that runs the result

Specification v0.2

F1R3FLY.io

22 September 2026

Abstract

Name-free combinators for the rho calculus (draft 3, [4]) repairs the combinator calculus of [9], supplies the rho-to-combinator translation the original promised and never gave, and proves it correct as full abstraction relative to encoded contexts. Its companion [5] gives the spatial-behavioural logic over the same calculus. Nothing executes either. The `campf1r3` workspace supplies, for kernel rho, everything a compiler of this shape needs on the source side — a parser and a normaliser for K0 [6], a store [3], an interpreter [2] — and nothing on the target side. This document specifies the arrow between them: five crates that take a K0 term to a combinator term, and run it.

Draft 3 now presents the target as an operational theory over two sorts and offers two sound presentations, A and B, differing in whether $*@p = p$ is an equation and hence in whether a message payload is a name or a process. A compiler cannot hold both: the term type, the store's datum type, the tag table and the decision procedure for $\equiv_{\mathcal{N}}$ all differ. Section 2 takes presentation A, for five reasons of which three are decisive — the logic is a logic for A and cannot tell a store from a message under B [5, Rem. 2.5]; quotation is injective only under A [5, Prop. 2.10], and CAMPF1R3 keys channels by the encoding; and the translation of [4, Def. 6.6] is written in A's notation. It also records that the implementation argument of [4, §4], which reads the F1R3FLY normaliser as evidence for B, does not survive contact with the code, and points the other way.

Two findings from v0.1 survive unchanged and are restated: [4, Def. 6.1]'s seed powers are constructible at run time by `consm`, so freshness fails against the calculus's own constructors (Erratum 5.4); and CAMPF1R3 keys channels by the encoding, so the compiler must be canonical, not merely correct up to bisimilarity, which Decision 5.6 secures. One decision from v0.1 is reversed: `q` *must* carry a process, not a name, because the sort asymmetry between `m` and `q` is the whole of what makes storage visible to the logic. Section 16 lists the changes. Key words MUST, MUST NOT, SHOULD, MAY are used in the sense of RFC 2119.

Contents

1	Purpose and scope	3
1.1	The gap	3
1.2	What exists today	4
1.3	Nomenclature	5
1.4	Goals	5
1.5	Non-goals	5
2	Which presentation	6
2.1	The choice cannot be deferred	6
2.2	Why A	6
2.3	The implementation argument in [4, §4]	7
2.4	The seam	8

3	The source language	8
3.1	K0 is the paper's rho	8
3.2	Two source presentations, agreeing where it matters	8
3.3	What the normaliser provides	9
4	The target calculus	9
4.1	Formation	9
4.2	Equations	10
4.3	Rules	10
4.4	Sorts: the $\mathbf{m}/\mathbf{q}$ asymmetry is not cosmetic	11
4.5	Representation, encoding, hashing	11
5	Canonicity	12
5.1	Why it is load-bearing	12
5.2	Seeds	12
5.3	The seed discipline	13
6	Architecture	14
6.1	The crates	14
6.2	The pipeline	14
6.3	Composing an executive	14
7	The lowering	15
7.1	Phases	15
7.2	Occurrence analysis	15
7.3	Distributor and gate	16
7.4	The clauses	16
7.5	Constructor chains	16
7.6	Shortcuts	17
7.7	Worked examples	17
7.8	Size	18
8	The machine	18
8.1	The target is store-shaped	18
8.2	The instantiation	18
8.3	Atoms as store operations	19
8.4	Name growth	19
9	The lattice audit, and what the logic needs	20
10	Correctness obligations and testing	20
11	Errors and diagnostics	21
12	Performance, portability, dependencies	22
13	What the implementation asks of the papers	22
14	Open questions and deferred work	23
15	Delivery	24
16	Changes from v0.1	25

1 Purpose and scope

1.1 The gap

Draft 3 ends with a translation $\llbracket - \rrbracket_s$ from the rho calculus into the repaired combinators, a release lemma that fixes the reduction sequence following a communication, and a full abstraction theorem relative to encoded contexts. The translation is defined on paper, by a table of five occurrence kinds and four clauses. It has never been run, and the properties that a paper translation is allowed to leave up to bisimilarity — which seed, which order of emission, which representative of a constructed name, and now which presentation — are properties an implementation is not allowed to leave open, because they decide whether two independently compiled terms can speak to one another.

On the other side, the `campf1r3` workspace (the repository `F1R3FLY-io/campf1r3`, crate family `f1r3-executive`) is a complete executive for kernel rho: `kindling-parse` and `kindling-norm` take text to a canonical normal form, `campf1r3-core` is the store, `b3ll0ws` interprets the normal form directly, and `f1r3x` drives them. It knows nothing of combinators.

F1R3COMB is the missing arrow, plus the smallest machine that can run its output.

1.2 What exists today

Artefact	What it gives this project
<code>publications/rho-combinators/rho-combinators-draft3</code>	The target calculus as an operational theory (§3.2–3.3, Def. 3.1, Def. 3.3), the two presentations (§3.3) and the argument between them (§4), the translation (Def. 6.6, Table 1), the correctness lemmas (§7), the expressiveness lattice (§9), the constructors-only presentation (§10)
<code>publications/rho-combinators/rhocomb-logic (draft 2)</code>	The logic over the combinators; the presentation decision (Dec. 2.3), what B costs the logic (Prop. 2.4, Rem. 2.5), drop elimination (Lem. 2.8), $\equiv_{\mathcal{N}}$ decided (Prop. 2.10), occupancy in linear time (Thm. 6.5)
<code>publications/campfir3/kindling-spec</code>	K0 = pure rho; the AST, the normal form, its encoding, its hashing, the congruence requirement (Def. 2.6, Req. 2.7, Prop. 6.9)
<code>publications/campfir3/campfir3-spec</code>	The store: D, W, I , produce/consume/install as single cuts, joins, profiles, replay
<code>publications/campfir3/bytecode-or-normal-form</code>	Why code must never be a channel key unless the compiler is canonical (§ identity); the backend seam
<code>campfir3 (the repository)</code>	<code>kindling-ast</code> , <code>kindling-parse</code> , <code>kindling-norm</code> , <code>campfir3-core</code> , <code>kindling-campfir3</code> , <code>b3ll0ws</code> , <code>f1r3x</code> , <code>f1r3x-ffi</code> , <code>f1r3x-wasm</code> ; no external dependencies at all

1.3 Nomenclature

Term	Meaning in this document
<i>source</i>	K0 of [6]: 0 , par, monadic send, monadic receive on a name variable, unquote, quote. The rho calculus of [4, §2], modulo Obligation 3.3
<i>target, the combinators</i>	The theory of [4, §3]: two sorts, thirteen atom shapes, 0 , par, quote and drop, no binders, no name hiding
<i>presentation A, B</i>	The two variants of [4, §3.3]; A adds $*@p = p$ and sorts message payloads as names, B omits it and sorts them as processes
<i>atom</i>	One combinator occurrence. e is an atom; $*-$ is a process former and is <i>not</i> e ([4, §3.2])
<i>message</i>	$m(a, v)$; the only loud value-bearing atom
<i>store</i>	$q(a, p)$, readable only by a forwarder
<i>router</i>	d, k, fw, b_l, b_r, s
<i>constructor</i>	cons, cons_m, cons_d, cons_s , and under the shortcuts feature cons_e, cons_{fw}
<i>seed</i>	A name s fresh for the term being translated; s^w its powers, indexed by a binary string
<i>proxy</i>	A constructed name c_i standing at one occurrence of a bound name in a released continuation
<i>link</i>	The atom emitted for one occurrence, per Table 1
<i>deploy</i>	Installing a compiled term into the machine's store

The product name is F1R3COMB, written **f1r3comb** in crate names and **comb** in F1R3x sub-commands. The calculus is *the rho combinators*, never *rhocomb*, which names the earlier draft [9].

1.4 Goals

1. A total, canonical, provably terminating compiler from K0 normal forms to combinator terms, implementing [4, Def. 6.6] with every degree of freedom in that definition closed — including the choice of presentation.
2. A target representation with its own canonical encoding and content hash, in the same style and the same tag-space discipline as K1NDLING's, so that $\equiv_{\mathcal{N}}$ on target names is a 32-byte comparison.
3. A machine that runs target terms by instantiating CAMPF1R3 — no spatial matcher, no substitution, no de Bruijn — with the same determinism and replay guarantees **campf1r3** already offers.
4. Differential testing against B3LL0WS: the same source program, run both ways, agreeing on the observations an encoded context can make.
5. A term type and an occupancy report that [5] can build on without a compiler.
6. The same portability envelope as the rest of the workspace: no external dependencies, **forbid(unsafe_code)**, WebAssembly-clean.

1.5 Non-goals

1. **The logic itself.** [5] specifies satisfaction, characteristic formulae, occupancy and a model checker. F1R3COMB MUST NOT implement any of them. It MUST expose the atom multiset and the lattice point (Section 9), which is what the occupancy checker needs from a compiler, and nothing more.

2. **K1 and K2.** Polyadic send and receive, joins, persistence, peek, **new**, name patterns and guards are outside the source language. The compiler **MUST** reject them with the level diagnostic of [6, Req. 2.1] rather than attempt a translation.
3. **The π route.** Composition with [8] is a later question.
4. **Decompilation.** No combinator-to-rho direction is specified.
5. **Cost accounting.** Out of scope; Section 8 leaves the CAMPF1R3 observer seam open so it can be added later.
6. **Optimisation beyond [4, Rem. 6.7].** The three shortcuts named there are the whole of the optimisation budget for v1.

2 Which presentation

2.1 The choice cannot be deferred

[4, §3.3] gives two presentations, both sound, and [4, Rem. 3.2] shows they cannot be mixed piecemeal. For a paper this is a pleasant degree of freedom: [4, Def. 3.3] records that no rule, lemma or proof in the note distinguishes them. For a compiler it is not a degree of freedom at all. Every artefact this specification names differs between them:

	Presentation A	Presentation B
m payload sort	N	P
q payload sort	P	P
eval	$e(a) \mid m(a, v) \longrightarrow *v$	$e(a) \mid m(a, p) \longrightarrow p$
*— in a target term	eliminable; absent from every normal form ([5, Lem. 2.8])	irreducible; needed, since $m(v, *x)$ is how a name is sent
@— injective	yes, on normal forms ([5, Prop. 2.10])	no ([5, Prop. 2.4(ii)])
$\equiv_{\mathcal{N}}$ decided by	byte equality of the canonical encoding	$@*a \rightarrow a$ normalisation, then comparison; strictly coarser than A's equality
Store datum type	Loud(CName) Quiet(Term)	Loud(Term) Quiet(Term)
Canonical form	flat multiset of atoms	multiset plus a descent production for $*\nu$
Logic support	full	needs a receptivity primitive before it can say anything about storage ([5, Rem. 2.5])

Decision 2.1 (F1R3COMB implements presentation A). The compiler, the term type, the encoding and the machine implement presentation A. B is a seam, not a default (Requirement 2.3).

2.2 Why A

1. **The logic needs it, and the logic is the other consumer of this term type.** [5, Dec. 2.3] states satisfaction against A and says the choice is forced rather than convenient: the grammar's $m(\nu_1, \nu_2)$ against $q(\nu, \phi)$ asymmetry exists only under A. Under B the two productions coincide, and by [5, Rem. 2.5] the logic cannot distinguish a store from a message *at all* — what separates them is a receptivity statement, which is not derivable from the context modality and negation, and is open item 5 of that note. Requirement 6.1 makes **f1r3comb-term** usable by the logic tooling without the compiler; a term type built for B would hand that tooling a problem it does not currently know how to solve.

2. **Channel identity requires injective quotation.** CAMPF1R3 keys a channel by the content hash of its encoding, and [6, Req. 2.7] requires equality of terms, of encodings and of hashes to agree. Under A, drop elimination ([5, Lem. 2.8]) is terminating and confluent, normal forms are drop-free, and $@-$ is injective on them ([5, Prop. 2.10]): equality of names is equality of bytes, full stop. Under B, $*@p$ and p are distinct processes with equal quotations, so two distinct payloads live at one key, and the agreement [6, Req. 2.7] asks for holds in one direction only. It is recoverable — normalise $@*a \rightarrow a$ and compare — but it is a second decision procedure where A has none.
3. **The translation is written in A.** [4, Def. 3.3] says payloads are written in A’s notation, $\mathbf{m}(a, @p)$, and §6 uses that notation throughout: $\llbracket x!(P) \rrbracket_s = \mathbf{m}(\llbracket x \rrbracket, @[\llbracket P \rrbracket]_{s0})$, the gate $\mathbf{q}(b, S)$ with a process payload, the constructor rules with $\mathbf{m}(a, @p)$ premises. Implementing A is transcription; implementing B is re-derivation of every clause, with the sort of each payload checked by hand.
4. **The representation is smaller.** No drop node in the term type, no descent production in any traversal, canonical forms that are flat multisets of atoms ([5, Prop. 2.4(iii)] is the statement of what B costs here).
5. **The existing normaliser already implements A on the source side.** `kindling_norm::Norm::eval` applies $*@P \rightarrow P$ at construction, and [6, Def. 2.6] states the source congruence with that clause in it. Choosing A makes one equation govern both sides of the compiler.

2.3 The implementation argument in [4, §4]

[4, §4] argues for B from the implementation: the F1R3FLY implementation normalises on decode, what is handed back has been rearranged and bound variables replaced by de Bruijn indices, neither transformation is derivable from the equations, so under A the implementation is unsound for its own theory and under B it is sound. The conclusion drawn is that the implementation is evidence for B.

That argument does not hold against the code, on three counts, and this specification records them because §4 is the paper’s principal argument for B and Decision 2.1 goes the other way.

1. **The rearrangement is the monoid laws.** What `kindling_norm` does to a parallel composition is sort its components by bitwise comparison of their encodings. Sorting a multiset is exactly commutativity and associativity of $|$, which [4, Def. 3.1] gives as *common* to both presentations. So this transformation is derivable from the equations, and the decoded process is equal to the quoted one, not merely bisimilar to it.
2. **De Bruijn indices are a source-calculus feature.** The combinators have no binders — [4, Def. 3.1] says so, and it is the point of the whole exercise. `kindling_norm` assigns de Bruijn indices to the binders of a kernel rholang `Receive`. A decoded *combinator* term has no bound variable to index, so the transformation never applies to the thing §4 is talking about.
3. **The one presentation-specific step the code performs is A’s equation.** `Norm::eval` rewrites $*@P$ to P . That is $*@p = p$, which A asserts and B denies. Under A it is a sound normalisation; under B it identifies two processes the theory keeps apart, and is the one place the implementation would be unsound.

Taken together, the evidence §4 appeals to points at A. §4’s structural argument — that decoding is an event under B and an identity under A, and that only B lets a decoder return a bisimilar representative — stands untouched; it is an argument about what a future implementation *may* do, not about what this one does.

Remark 2.2 (What A gives up, and what it does not). A forfeits the decoder’s freedom to return an optimised representative. The concrete thing that freedom would buy is a backend that discharges the administrative forwarders of the link expansion ([4, Lem. 7.4]) at decode time, handing back a smaller bisimilar term. Nothing in `campf1r3` does this today, and B3LL0WS’s `Direct` backend substitutes into the normal form and takes its top-level components without optimising. Should such a backend be wanted, it needs B, and that is the one development to watch. A does *not* cost anything in cost accounting: `eval` is a reduction step under both presentations and is equally chargeable; what A fixes is only the identity of the result, not whether producing it is an event.

2.4 The seam

Requirement 2.3 (Presentation is a compile-time parameter). The presentation **MUST** be a cargo feature, `presentation-b`, off by default, and the crates **MUST** be structured so that turning it on changes the payload sort of `m`, the `eval` rule, the presence of the drop node and the $\equiv_{\mathcal{N}}$ decision procedure, and nothing else. It **MUST** be recorded in the header of every compiled artefact, and a deploy **MUST** diagnose a mismatch (`comb-presentation-mismatch`) rather than proceed: under B the same source compiles to a term with different bytes and a different hash, so two deploys that disagree about the presentation cannot meet, and must be told so.

Requirement 2.4 (No piecemeal mixing). [4, Rem. 3.2] shows that omitting $*@p = p$ while keeping name-sorted payloads leaves `eval` either stuck or ambiguous. The build **MUST** therefore couple the equation, the payload sort and the `eval` rule in one feature, and **MUST NOT** expose them as three. A test **MUST** assert that the feature matrix has exactly two reachable configurations.

3 The source language

3.1 K0 is the paper’s rho

[4, Def. 2.1]	K0 concrete syntax	<code>kinding-norm</code> Node
0	<code>Nil</code>	<code>Nil</code>
$P \mid Q$	<code>P Q</code>	<code>Par(Vec<Norm>)</code> , sorted, no <code>Nil</code>
$x!(P)$	<code>x!(P)</code>	<code>Send{chan, persistent:false, args:[P]}</code>
$\text{for}(y \leftarrow x)P$	<code>for(y <- x){P}</code>	<code>Receive{binds:[b], body}</code>
$*x$	<code>*x</code>	<code>Eval(Name)</code>
$@P$	<code>@P</code>	<code>Name::Quote(Norm)</code>

Requirement 3.1 (Source level). `F1R3COMB` **MUST** accept exactly K0, invoking `kinding-parse` with `Level::K0`. A `Receive` whose bind list has other than one bind, a bind with other than one pattern, a pattern that is not a name variable, a `Send` with other than one argument, a persistent bang or arrow, a peek, a `New` node, an `Unforgeable` name, a `Wild`, a `FreeVar` or a `FreeName` in a pattern position **MUST** each produce a distinct diagnostic (Section 11).

Requirement 3.2 (Closed input). The term presented to the compiler **MUST** be closed: no `FreeVar`, no `FreeName`. `Name::Unforgeable` **MUST** be rejected at `v1`, because the target has no name former for it and inventing one would enlarge the atom set outside [4] and outside $\mathcal{A}_{\text{full}}$ as [5, Def. 2.1] counts it. Grounding **MUST** therefore be by quotation, and the diagnostic **MUST** say so.

3.2 Two source presentations, agreeing where it matters

[4, §2] states the source congruence *without* $*@P \equiv P$ and recovers its work through semantic substitution ([4, Def. 2.3]). [6, Def. 2.6] states it *with* that clause, and `Norm::eval` applies it

eagerly. The two source calculi are therefore not the same calculus, and F1R3COMB compiles the second while claiming the theorems of the first.

Obligation 3.3 (The source presentations agree on guarded-drop terms). For P guarded-drop in the sense of [4, Def. 2.5] — every $*x$ has x bound by an enclosing input — no subterm $*@Q$ occurs in P or in any reduct of P ([4, Lem. 2.6]), so the rewrite $*@P \rightarrow P$ never fires and the two congruences coincide on P . This **MUST** be proved, and it is what licenses compiling a K1NDL1NG normal form against [4, §2]. It is also why nobody has noticed the mismatch.

Requirement 3.4 (Guarded-drop check, now a diagnostic). The compiler **MUST** verify the guarded-drop property on the *tree*, before normalisation, and **MUST** report **comb-drop-quote** for a literal $*@Q$ and **comb-unguarded-drop** for a drop of any other unbound name. Under Decision 2.1 these are warnings, not errors, and compilation **MAY** proceed: the source normaliser and the target theory both satisfy $*@p = p$, so the collapsed term is equal, not merely bisimilar, to the written one. Under **presentation-b** they **MUST** be errors, because there the collapse is unsound and Obligation 3.3 is the only thing standing between the compiler and a source term the theory does not describe.

Remark 3.5 (v0.1 had this as a hazard). v0.1 made `Norm::eval` the sharpest edge in the specification, on the grounds that it applies an equation the target must not have. Under Decision 2.1 the target *does* have that equation, and the edge is gone: one rewrite, sound on both sides, applied by one function. The edge returns in full under **presentation-b**, which is the substance of Requirement 4.5.

3.3 What the normaliser provides

Property	Use here
$\llbracket P \rrbracket = \llbracket Q \rrbracket \iff P \equiv Q$ ([6, Prop. 6.9])	The input is a canonical representative of a congruence class, which is what makes Obligation 5.8 possible
Par components sorted by their own encodings	A canonical path index w for seed allocation (Section 5)
$@*x \rightarrow x$, in <code>Norm::quote</code>	The equation common to both presentations and to both calculi
$*@P \rightarrow P$, in <code>Norm::eval</code>	A's equation; see Obligation 3.3
de Bruijn indices for bound names	Occurrence analysis becomes an index comparison (Section 7.2)
BLAKE2b-256 of the encoding	Channel identity, reused verbatim for target names

Decision 3.6 (Compile the normal form, not the tree). The compiler's input is `k1ndl1ng_norm::Norm`, with the sole exception of the guarded-drop check of Requirement 3.4, which needs the tree. Compiling the tree would make the output depend on the order in which the author happened to write a parallel composition, and Obligation 5.8 would be false.

4 The target calculus

4.1 Formation

From [4, §3.2], with $a, b, c, f : \mathbf{N}$ and $p, q : \mathbf{P}$; every atom has sort $\mathbf{P}$ and every argument is a name except the second argument of $\mathbf{q}$ and, under $\mathbf{B}$, of $\mathbf{m}$.

$m(a, v)$	$b_l(a, b)$	$\text{cons}(a, b, c)$
$d(a, b, c)$	$b_r(a, b)$	$\text{cons}_m(a, b, c)$
$k(a)$	$s(a, b, c)$	$\text{cons}_d(a, b, c, f)$
$\text{fw}(a, b)$	$e(a)$	$\text{cons}_s(a, b, c, f)$
$q(a, p)$	$\mathbf{0}, p \mid q$	$@p : \mathbf{N}, *a : \mathbf{P}$

Requirement 4.1 (The drop and the guard are distinct constructors). $*-$ and e MUST be distinct constructors of the target type, and no code path MAY convert one into the other. This is the whole of the repair: [4, Prop. 3.5] needs the conflation as a hypothesis, and with the constructors separate an arbitrary running process cannot match a guard however many equations the theory carries. The type system enforces it for free provided $*-$ is not represented as a nullary-argument e or vice versa; a representation sharing one tag between them MUST NOT be used even with a discriminating flag.

Requirement 4.2 (Rigidity, as a test). No equation of Definition 4.3 has e at its head, and the rewrite system of Requirement 4.4 preserves head constructors other than $*-$. A test MUST assert the consequence: for every P in a corpus and every name a , the normal form of P is an e atom only if P was written as one. This is Requirement 4.1 in executable form.

4.2 Equations

Definition 4.3 ([4] Def. 3.1, with A's addition). $(\mathbf{P}, |, \mathbf{0})$ is an abelian monoid; $@*a = a$; and, under presentation A, $*@p = p$.

Requirement 4.4 (Drop elimination). Orient the equations left to right: $*@p \rightarrow p$ and $@*a \rightarrow a$. By [5, Lem. 2.8] the system is terminating and confluent and the normal form of a closed term contains no $*-$. The smart constructors of `f1r3comb-term` MUST apply both rewrites at construction, exactly as `Norm::eval` and `Norm::quote` do for the source, so that every `Term` in existence is in normal form and $*-$ appears only as a transient argument to a constructor. Every traversal in the crate MAY therefore assume a flat multiset of atoms.

Requirement 4.5 (Under B, the opposite). With `presentation-b` enabled, $*@p \rightarrow p$ MUST NOT be applied, the drop node MUST be retained in the normal form, every traversal MUST carry a descent case for it ([5, Prop. 2.4(iii)]), and the term type MUST NOT be routed through `kindling_norm::Norm::eval` or any function that calls it. The `presentation-b` build MUST carry a test asserting that $*@p$ and p have different encodings.

4.3 Rules

Definition 4.6 ([4] Def. 3.3). Payloads written in A's notation; under B read $m(a, p)$ for $m(a, @p)$.

$\text{dist} : d(a, b, c) \mid m(a, v) \longrightarrow m(b, v) \mid m(c, v)$
 $\text{kill} : k(a) \mid m(a, v) \longrightarrow \mathbf{0}$
 $\text{forward} : \text{fw}(a, b) \mid m(a, v) \longrightarrow m(b, v)$
 $\text{bind}_l : b_l(a, b) \mid m(a, v) \longrightarrow \text{fw}(v, b)$
 $\text{bind}_r : b_r(a, b) \mid m(a, v) \longrightarrow \text{fw}(b, v)$
 $\text{synch} : s(a, b, c) \mid m(a, v) \longrightarrow \text{fw}(b, c)$
 $\text{eval} : e(a) \mid m(a, v) \longrightarrow *v \quad (\text{A}); \quad e(a) \mid m(a, p) \longrightarrow p \quad (\text{B})$
 $\text{quote} : \text{fw}(a, b) \mid q(a, p) \longrightarrow m(b, @p)$
 $\text{make}_l : \text{cons}(a, b, c) \mid m(a, @p) \mid m(b, @r) \longrightarrow m(c, @(p \mid r))$
 $\text{make}_m : \text{cons}_m(a, b, c) \mid m(a, u) \mid m(b, v) \longrightarrow m(c, @m(u, v))$
 $\text{make}_d : \text{cons}_d(a, b, c, f) \mid m(a, u) \mid m(b, v) \mid m(c, w) \longrightarrow m(f, @d(u, v, w))$
 $\text{make}_s : \text{cons}_s(a, b, c, f) \mid m(a, u) \mid m(b, v) \mid m(c, w) \longrightarrow m(f, @s(u, v, w))$

closed under parallel composition and under the equations, with subjects matched up to $\equiv_{\mathcal{N}}$.

Requirement 4.7 (The rule table is the implementation). The rules **MUST** be implemented as data — one table, one row per atom shape, giving arity, which arguments are consumed subjects, which are produced subjects, and how the result is assembled — and the machine’s step function **MUST** be a dispatch over that table. No rule may be special-cased in control flow. A reviewer comparing the table against Definition 4.6 line by line is the primary correctness argument at this layer.

Requirement 4.8 (The three departures are load-bearing). [4, Rem. 3.4] records three corrections that are easy to reintroduce, and the implementation **MUST** carry a test for each. The binders produce forwarders, not messages: $\mathbf{b}_l(a, b) \mid \mathbf{m}(a, v) \longrightarrow \mathbf{fw}(v, b)$ and $\mathbf{b}_r(a, b) \mid \mathbf{m}(a, v) \longrightarrow \mathbf{fw}(b, v)$. A version producing a message duplicates **forward**, collapses the lattice edge the observational ordering needs, and breaks occurrence kinds (o1) and (o2) of Table 1, which require a redirection and not a value. **fw** is binary. And a constructor’s delivery channel f is in its own argument list and distinct from its input channels — which Requirement 7.10 strengthens.

4.4 Sorts: the $\mathbf{m}/\mathbf{q}$ asymmetry is not cosmetic

Decision 4.9 ($\mathbf{q}$ carries a process; under $\mathbf{A}$, $\mathbf{m}$ carries a name). The store’s payload has sort $\mathbf{P}$ and the message’s, under $\mathbf{A}$, has sort $\mathbf{N}$. The datum type is therefore a two-case enum and not a name with a flag.

Remark 4.10 (This reverses v0.1). v0.1 took [4, Rem. 3.7]’s observation that writing the store’s payload as a name makes the parallel with $\mathbf{m}$ exact, and made it a decision, on the grounds of uniformity: every atom taking only names, no process-in-atom case in the encoding. [5, Rem. 2.5] shows what that uniformity costs. The sort asymmetry between $\mathbf{m}$ and $\mathbf{q}$ is precisely what makes a store structurally distinguishable from a message in the logic; erase it and the only remaining difference is which rules consume them, which is a receptivity statement, which the logic cannot express. The v0.1 decision would have handed [5] the B problem while nominally implementing A. It is withdrawn.

Requirement 4.11 (Sorts are types). $\mathbf{N}$ and $\mathbf{P}$ **MUST** be distinct Rust types, and the atom constructors **MUST** take them positionally as Definition 4.3 sorts them. A single untyped node array with an arity field **MUST NOT** be used: the sorting is what [5, Rem. 2.6] is asking the paper for and what the logic’s grammar reads off.

4.5 Representation, encoding, hashing

Requirement 4.12 (Encoding). A target term **MUST** have a canonical byte encoding built bottom-up in the smart constructors and cached on the node, in the style of [6, §6]: tag byte, then LEB128 lengths and counts, then children. Parallel components **MUST** be ordered by bitwise lexicographic comparison of their own encodings, with $\mathbf{0}$ absent. The content hash **MUST** be BLAKE2b-256 of the encoding, computed by `kindling_norm::hash`.

Requirement 4.13 (Tag disjointness). Target tags **MUST** occupy a block disjoint from the source tags of [6, §6] ($\mathbf{0x00}$ – $\mathbf{0x20}$), so that no target encoding is a valid source encoding and a confusion is a decode error rather than a silent misread.

Tag	Shape	Tag	Shape
0x30	0	0x39	$q(a, p)$, payload a term
0x31	par, $n \geq 2$ components	0x3A	$\text{cons}(a, b, c)$
0x32	$m(a, v)$	0x3B	$\text{cons}_m(a, b, c)$
0x33	$d(a, b, c)$	0x3C	$\text{cons}_d(a, b, c, f)$
0x34	$k(a)$	0x3D	$\text{cons}_s(a, b, c, f)$
0x35	$\text{fw}(a, b)$	0x3E	$\text{cons}_e(a, b)$, shortcuts
0x36	$\text{b}_l(a, b)$	0x3F	$\text{cons}_{\text{fw}}(a, b, c)$, shortcuts
0x37	$\text{b}_r(a, b)$	0x40	$@p$, name former
0x38	$s(a, b, c)$	0x41	$*a$, presentation-b only
		0x42	$e(a)$

Requirement 4.14 (The drop tag is unreachable under A). **0x41** MUST be rejected by the decoder in the default build, since by Requirement 4.4 no normal form contains a drop. A byte string carrying it is either a **presentation-b** artefact — diagnosed by Requirement 2.3 — or corrupt. Reserving the tag rather than reusing it keeps the two encodings comparable.

Requirement 4.15 (Name equivalence is a hash comparison). Under A, for target names a, b : $a \equiv_{\mathcal{N}} b$ iff their encodings are equal iff their content hashes are equal modulo collision. This is [5, Prop. 2.10], and the machine MUST match subjects by content hash and MUST NOT implement a separate $\equiv_{\mathcal{N}}$ procedure. It discharges corrigendum 5 of [4, §11]. Under **presentation-b** the comparison MUST be preceded by $@*a \rightarrow a$ normalisation and the crate MUST document that the resulting relation is strictly coarser than encoding equality.

Requirement 4.16 (No target variables). The target node type MUST NOT have a bound-variable, free-variable or wildcard case. There are no binders in the target; a representation that can express one can express a term the compiler must never emit.

5 Canonicity

5.1 Why it is load-bearing

[2, § identity] states the constraint in its source-side form: CAMPF1R3’s channel identity is the encoding, so a compiled form may not be a channel key unless the compiler is proved canonical. In F1R3COMB the constraint binds hard, because compiled code is always a key: the translation of a source name is $\llbracket @P \rrbracket = @ \llbracket P \rrbracket_s$, the quotation of compiled code, and that name is what two compiled terms must agree on if a message is ever to meet a receiver.

Requirement 5.1 (Canonicity). For source terms P, Q : if $P \equiv Q$ then the encodings of $\llbracket P \rrbracket$ and $\llbracket Q \rrbracket$ are byte-equal. Compilation MUST be a function of the source normal form alone — not of the source text, not of the order of traversal, not of a counter, not of the time, not of the deploy.

Remark 5.2 (The paper gives less than this, and says so). [4, Lem. 7.8] gives seed independence: $\llbracket P \rrbracket_s \approx \llbracket P \rrbracket_t$ for any two seeds. Bisimilar is not equal. Two deploys that compile the same source with different seeds produce terms that behave alike and hash differently, so their channels differ and their messages never meet.

5.2 Seeds

Definition 5.3 ([4] Def. 6.1, amended). A *seed* for P is a name $s \notin \mathcal{N}^*(P)$ modulo $\equiv_{\mathcal{N}}$. Set $s^e = s$ and, for a binary string w ,

$$s^{w0} = @b_l(s^w, s^w), \quad s^{w1} = @k(s^w).$$

Erratum 5.4 (The paper’s seed powers are constructible). [4, Def. 6.1] sets $s^{w0} = @m(s^w, @0)$. That name lies in the image of cons_m : the rule make_m manufactures $@m(u, v)$ from whatever arrives at a and b , so a term that receives s^w and $@0$ can build s^{w0} at run time. The freshness [4, Lem. 6.2] establishes is freshness against $\mathcal{N}^*(P)$, the names *occurring* in P ; with constructors in the calculus the relevant property is freshness against the names P can *reach* — the closure namespace $\text{Cl}_S(T)$ of [5, Def. 8.1] — and the two differ. Definition 5.3 repairs it: the constructor family builds quotations of $|$, m , d and s nodes only, so a seed power quoting a b_l or a k node is outside the image by inspection of the tag. The two constructors remain injective with disjoint images, so [4, Lem. 6.2] goes through verbatim, and the encoded-observer restriction is no longer doing freshness work it was not designed to do.

Requirement 5.5 (Seed powers stay outside the constructor image). The seed constructors **MUST** quote atom shapes not in the image of any enabled constructor. With **shortcuts** enabled, cons_e and cons_{fw} add $@e(-)$ and $@fw(-, -)$ to that image; b_l and k remain outside it. Any future constructor **MUST** be checked against Definition 5.3, and the crate **MUST** carry a test asserting disjointness by tag for the enabled feature set. The test is the machine form of the closure fixpoint of [5, Prop. 8.2].

5.3 The seed discipline

[4, Def. 6.6] says: on names, $[[@P]] := @[P]_{s^w}$ where w is the position of that quotation in the enclosing term. Read literally, the same source name occurring at two positions compiles to two different target names, and two terms that communicate on $@P$ in the source do not communicate at all in the target. The sentence is a slip for a paper whose results are stated up to $\approx$; it is fatal for a compiler.

Decision 5.6 (Term-determined seeds). The seed for a quotation is the quotation itself:

$$[[@P]] := @[P]_{@P}.$$

[4, Lem. 6.2] licenses the choice — “ $s := @P$ is a seed for P ” — and it is the unique choice that is a function of the name alone. Seed powers w are allocated by path within one translation unit, where a unit is the body of one quotation; w never crosses a quotation, because compiling a quotation resets the seed.

Requirement 5.7 (Canonical paths). Within a translation unit, the path w assigned to a node **MUST** be read off the source normal form. $\text{Norm}::\text{Par}$ is n -ary and [4, Def. 6.6] is binary, so the binarisation is fixed here as the right-leaning spine: for components in encoding order $P_0, \dots, P_{n-1}$, component i receives the path extension 1^i0 for $i < n - 1$ and 1^{n-1} for $i = n - 1$. For a send, the payload receives 0; for a receive, the body receives 0 and the links 1, 10, 100, $\dots$ in the order fixed by Requirement 7.3. The allocation **MUST** be a pure function of the normal form and **MUST NOT** consult a mutable counter whose value depends on traversal order elsewhere in the term.

Obligation 5.8 (Canonicity theorem). With Decision 5.6 and Requirements 5.7 and 7.3, compilation is a function of the source normal form, and the source normal form is a function of the source congruence class ([6, Prop. 6.9]). Hence $P \equiv Q$ implies $\text{enc}([P]) = \text{enc}([Q])$. This **MUST** be proved and tested (Obligation 10.1). Under A the proof is shorter than it was in v0.1: target normal forms are drop-free flat multisets (Requirement 4.4), so the only quotient to reason about is the multiset ordering.

Remark 5.9 (Seed size). [4, Rem. 6.3] notes that [9] builds each constructed name by quoting the subterm it must be fresh for, making the output quadratic in $|P|$. Decision 5.6 reintroduces a quotation of the source name as the seed — but only one, shared by every power within the unit, and the powers grow as $|s| + O(|w|)$. With hash-consing (Requirement 8.8) the shared prefix is stored once.

6 Architecture

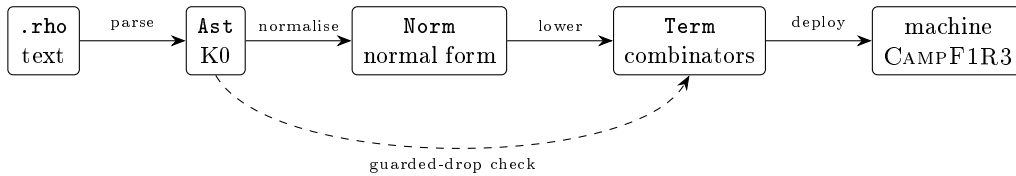
6.1 The crates

Five crates, each depending only on the ones above it, in the workspace `campfir3`. No external dependencies; `forbid(unsafe_code)` throughout.

Crate	What it is	Depends on
<code>f1r3comb-term</code>	The target: sorts, atom enum, smart constructors with drop elimination, encoding, hashing, printer, decoder, atom multiset	<code>kindling-norm</code> (hash only)
<code>f1r3comb-lower</code>	The translation: level and guard checks, seed allocation, occurrence analysis, constructor chains, emission	<code>kindling-ast</code> , <code>kindling-norm</code> , <code>f1r3comb-term</code>
<code>f1r3comb-space</code>	CAMPF1R3 instantiated over target terms: channel, datum, pattern, continuation, matcher	<code>campfir3-core</code> , <code>f1r3comb-term</code>
<code>f1r3comb-run</code>	The machine: deploy, the rule table, the scheduler, the event stream	<code>f1r3comb-space</code>
<code>f1r3comb-check</code>	Differential testing against B3LL0WS, lattice audit, golden files	all of the above, <code>b3ll0ws</code>

Requirement 6.1 (Independent usability). `f1r3comb-term` MUST be usable without `f1r3comb-lower`, because the occupancy checker of [5, §6] wants a term type, an encoding and an atom multiset and no compiler; and `f1r3comb-lower` MUST be usable without `f1r3comb-run`. No crate MAY depend on `f1r3comb-check`.

6.2 The pipeline



The dashed arc is Requirement 3.4: the check reads the tree, not the normal form, because `Norm::eval` has already applied $*@P \rightarrow P$ by the time the normal form exists.

6.3 Composing an executive

F1R3x gains four subcommands, alongside `run`, `show` and `hash`:

<code>f1r3x comb file</code>	Compile and print the combinator term
<code>f1r3x comb-hash file</code>	The content hash of the compiled term
<code>f1r3x comb-run file</code>	Compile and run; <code>-cores</code> , <code>-trace</code> , <code>-free-running</code> as for <code>run</code>
<code>f1r3x comb-lattice file</code>	The atom multiset and the lattice point of Section 9

	occurrence of y	in the quoted body	link
(o1)	input subject $\text{for}(z \leftarrow y)S$	subject $\rightarrow c_i$	$\mathbf{b}_l(p_i, c_i)$
(o2)	output subject $y!(S)$	subject $\rightarrow c_i$	$\mathbf{b}_r(p_i, c_i)$
(o3)	forwarded name $v!(*y)$	atom deleted	$\mathbf{fw}(p_i, \llbracket v \rrbracket)$
(o4)	drop $*y$ in process position	$\rightarrow \mathbf{e}(c_i)$	$\mathbf{fw}(p_i, c_i)$
(o5)	inside a quotation $@S, y$ free in S	subject $\rightarrow c_i$	constructor chain for S

Table 1: The five occurrence kinds, [4, Table 1].

Requirement 6.2 (Core-count invariance carries over). **comb-run** MUST reproduce the property **run** already has: with the store in its deterministic profile and a FIFO queue over encoding-ordered components, the final scene MUST be identical at one core, at sixty-four, and free-running.

7 The lowering

7.1 Phases

1. **Level check** on the tree (Requirement 3.1).
2. **Guard check** on the tree (Requirement 3.4): warnings under A, errors under B.
3. **Normalise** (**kindling-norm**).
4. **Closedness check** on the normal form (Requirement 3.2).
5. **Lower**, one translation unit at a time, bottom-up, memoised on the source name (Section 7.4).
6. **Encode** and hash (Requirement 4.12).

Requirement 7.1 (Total and terminating). Lowering MUST be total on terms that pass phases 1, 3 and 4, MUST terminate, and MUST NOT recurse on the Rust stack: the workspace’s standing rule — an explicit stack, no recursion anywhere including **Drop** — applies here as it does to **kindling-parse**.

7.2 Occurrence analysis

Requirement 7.2 (Classification). For a receive with body B , the analysis MUST walk B and collect every **Name::Bound**(d) whose de Bruijn index equals the number of binders between it and the receive, and MUST assign each exactly one kind by the following disjoint test on its immediate syntactic role:

- (a) the **chan** of a **Receive** bind, not under a quotation between it and the receive: **o1**;
- (b) the **chan** of a **Send**, not under a quotation: **o2**;
- (c) the name of an **Eval** which is the sole argument of a **Send** whose channel is not the occurrence, not under a quotation: **o3**;
- (d) the name of an **Eval** in process position, not under a quotation: **o4**;
- (e) anywhere under a **Name::Quote** lying between the occurrence and the receive: **o5**, with the chain built for the outermost such quotation.

The tests **MUST** be applied in this order, so that (c) takes precedence over (d). An occurrence matching none **MUST** raise an internal error naming the node, never a silent default.

Requirement 7.3 (Emission order). Occurrences **MUST** be enumerated by ascending byte offset of their occurrence in the encoding of the receive’s body. This order is a function of the normal form and is therefore canonical, which is what Requirement 5.7 needs. [4, Def. 6.6] says only “let $o_1, \dots, o_k$ enumerate the free occurrences” and leaves the order open; the order decides which proxy is c_1 , hence which seed power each link gets, hence the output bytes.

Requirement 7.4 (Coincident kinds). [4, Table 1] notes that when several kinds coincide in one atom, the atom is deleted from the body and the links are emitted together. Each *occurrence* has exactly one kind by Requirement 7.2; what coincides is several occurrences in one atom, as in $y!(*y)$. The implementation **MUST** treat them independently, each with its own proxy and link, in the order of Requirement 7.3, and **MUST NOT** fuse two occurrences into one proxy even when the result would be smaller.

7.3 Distributor and gate

Definition 7.5 ([4] Def. 6.4). For b, c fresh, $G(p, S) := s(p, b, c) \mid q(b, S) \mid e(c)$, with S a process by Decision 4.9; and $D(x; p_0) := \mathbf{0}$ with $p_0 := x$, while for $k \geq 1$

$$D(x; p_0, \dots, p_k) := d(x, p_0, q_1) \mid d(q_1, p_1, q_2) \mid \dots \mid d(q_{k-1}, p_{k-1}, p_k).$$

Requirement 7.6 (Gate and distributor names). b, c , the p_i and the q_j **MUST** be seed powers of the unit’s seed, allocated by Requirement 5.7. The degenerate case $k = 0$ **MUST** emit no duplicator and take $p_0 := \llbracket x \rrbracket$ directly, as Definition 7.5 requires; emitting a one-way d instead would be correct up to $\approx$ and would break Obligation 5.8 against a compiler that did not.

7.4 The clauses

Definition 7.7 ([4] Def. 6.6, as implemented). On names, $\llbracket @P \rrbracket := @\llbracket P \rrbracket_{@P}$ (Decision 5.6). On processes,

$$\begin{aligned} \llbracket \mathbf{0} \rrbracket_s &= \mathbf{0} \\ \llbracket P_0 \mid \dots \mid P_{n-1} \rrbracket_s &= \llbracket P_0 \rrbracket_{s^0} \mid \llbracket P_1 \rrbracket_{s^{10}} \mid \dots \mid \llbracket P_{n-1} \rrbracket_{s^{1^{n-1}}} \\ \llbracket x!(P) \rrbracket_s &= m(\llbracket x \rrbracket, @\llbracket P \rrbracket_{s^0}) \\ \llbracket \text{for}(y \leftarrow x)P \rrbracket_s &= D(\llbracket x \rrbracket; p_0, \dots, p_k) \mid G(p_0, B) \mid L_1 \mid \dots \mid L_k \end{aligned}$$

where $o_1, \dots, o_k$ enumerate the free occurrences of y in P in the order of Requirement 7.3, each o_i has proxy c_i , L_i and the residue in B are given by Table 1, and B is $\llbracket P \rrbracket_{s^0}$ with every occurrence treated accordingly.

Requirement 7.8 (Memoisation on the name, not the position). Lowering **MUST** memoise translation units on the source name $@P$ — equivalently, on the content hash of the source normal form P — so that a name occurring at several positions is compiled once and yields one target name. This is Decision 5.6 in operational form; a position-keyed cache instead would make the compiler non-canonical.

7.5 Constructor chains

Requirement 7.9 (Chain construction). For an occurrence of kind (o5) inside the outermost enclosing quotation $@S$, the compiler **MUST** emit the chain rebuilding $@\llbracket S \rrbracket$ from the received name and delivering it at c_i , by recursion on the path from the occurrence to the root of S , per [4, Prop. 3.9]:

- at the occurrence, the value is the received name; no atom;

- at $S' \mid S''$, emit $\text{cons}(a, b, c)$ with the dynamic side's output at its position and a constant message for the static side;
- at $v!(T)$, emit cons_m , covering a dynamic subject, a dynamic payload, or both;
- at $\text{for}(z \leftarrow v)T$, emit cons_d when z occurs in T and cons_s when it does not, matching the head atom of the image of the input under Definition 7.7;
- every node off the path is static and is supplied as a constant message.

Chains **MUST** be built outermost-last, so each constructor's inputs are available before it fires.

Requirement 7.10 (No repeated subject in a constructor). A constructor **MUST NOT** be emitted with two equal argument subjects, nor with a delivery channel equal to an input channel ([4, Rem. 3.4]). Where the same value is needed twice — as in Example 7.14, where both sides of a **cons** want the received name — the compiler **MUST** split first with **d**. The store's join is over a *group* of channels; a group with a repeated channel is not a request for two data at that channel and **MUST** not be issued as one.

Requirement 7.11 (Chain sharing). Where several (o5) occurrences lie under one quotation, the compiler **SHOULD** share the static constant messages and the common prefix of their chains, splitting with **d** at the deepest common node. Sharing **MUST** be deterministic, hence canonical, and **MUST** be disabled by **-no-share** for differential testing against the unshared form.

7.6 Shortcuts

Requirement 7.12 (The shortcuts are feature-gated, and cost two atoms). [4, Rem. 6.7] gives three cheaper special cases: $\llbracket \text{for}(y \leftarrow x)\mathbf{0} \rrbracket = k(\llbracket x \rrbracket)$, $\llbracket \text{for}(y \leftarrow x)*y \rrbracket = e(\llbracket x \rrbracket)$, and $\llbracket \text{for}(y \leftarrow x)v!(*y) \rrbracket = \text{fw}(\llbracket x \rrbracket, \llbracket v \rrbracket)$ for $v \neq y$. They **MUST** be behind the **shortcuts** feature, off by default, because [4, Rem. 3.10] records that taking them puts a bound name into the argument of an **e** or a **fw**, so the translation then needs cons_e and cons_{fw} . Enabling the feature enlarges the atom set from thirteen shapes to fifteen, moves the lattice point, and changes every content hash. It **MUST** be recorded in the artefact header alongside the presentation.

7.7 Worked examples

Example 7.13 ([4] Ex. 6.8). $P = \text{for}(y \leftarrow a)y!(\mathbf{0})$ has one occurrence, of kind (o2), so

$$\llbracket P \rrbracket_s = d(a, p_0, p_1) \mid G(p_0, m(c, @0)) \mid b_r(p_1, c).$$

Supplying $m(a, @\llbracket R \rrbracket)$ gives $m(p_0, @\llbracket R \rrbracket) \mid m(p_1, @\llbracket R \rrbracket)$; then $m(c, @0) \mid \text{fw}(c, @\llbracket R \rrbracket)$; then $m(@\llbracket R \rrbracket, @0) = \llbracket @R!(\mathbf{0}) \rrbracket$. Note that b_r yields a forwarder, per Requirement 4.8.

Example 7.14 (The witness, [4] Ex. 6.9). $W = \text{for}(y \leftarrow a)@(*y \mid *y)!(\mathbf{0})$ has one occurrence of kind (o5), so

$$\llbracket W \rrbracket_s = d(a, p_0, p_1) \mid G(p_0, m(c, @0)) \mid d(p_1, r_1, r_2) \mid \text{cons}(r_1, r_2, c).$$

The inner duplicator is Requirement 7.10 in action. Receiving $m(a, @\llbracket R \rrbracket)$ builds $m(c, @(\llbracket R \rrbracket \mid \llbracket R \rrbracket))$.

Requirement 7.15 (Both examples are golden tests). Examples 7.13 and 7.14 **MUST** be checked into **f1r3comb-check** as golden files: source, expected term, expected encoding, expected content hash, and the expected reduction sequence with step counts, for each presentation. They are the two the paper works by hand.

7.8 Size

Requirement 7.16 (Output size). Let k_r be the number of occurrences of the bound name in receive r , and for an (o5) occurrence o let d_o be the path length from o to the root of its enclosing quotation. The compiler **MUST** produce a term of size

$$||P|| \leq c \left(|P| + \sum_r k_r + \sum_{o \in o5} d_o \right)$$

and **MUST NOT** exhibit the quadratic name growth of [9] ([4, Rem. 6.3]). On the nesting family of [4, Prop. 8.2] the output **MUST** be $3d + 1$ atoms against Yoshida’s $(3^{d-1} + 1)/2$, tested as a law parameterised over d .

8 The machine

8.1 The target is store-shaped

Every rule of Definition 4.6 has the same form: consume a message at each of one, two or three named subjects, and produce messages or install atoms. There is no pattern to match beyond the subject, no binder to open, no substitution to perform. **CAMPF1R3** is exactly a store of that shape, with joins for the multi-subject case.

Decision 8.1 (The machine is **CAMPF1R3** with a three-line matcher). **f1r3comb-run** **MUST** be **campf1r3-core::Space** instantiated over target terms, and **MUST NOT** implement its own indexing, joins or replay. The spatial matcher of **kindling-campf1r3** is not used and **MUST** not be linked. The result is materially smaller than **B3LL0WS**: no substitution, no plan cache, no de Bruijn, no prepared continuations.

8.2 The instantiation

CAMPF1R3 parameter	F1R3COMB instance
C , channel	CName , a target name; content_hash is BLAKE2b-256 of its encoding (Requirement 4.15)
A , datum	enum Val { Loud(CName), Quiet(Term) } under A — the sorts differ, per Decision 4.9, so this is an enum over two types and not a struct with a flag; under B , Loud(Term)
P , pattern	Acc { accepts_quiet: bool } : two values, one for fw and one for everything else
K , continuation	Rule , the atom minus its consumed subjects
M , matcher	get(p,a) = match a { Quiet(_) if !p.accepts_quiet => None, _ => Some(a.clone()) }
O , observer	NoObserver at v1 ; the seam where cost accounting attaches

Requirement 8.2 (The loud/quiet distinction lives in the matcher). **q(a,p)** is a message only a forwarder may read ([4, Rem. 3.7]). This **MUST** be realised as the matcher rejecting a quiet datum for a non-forwarder pattern, and **MUST NOT** be realised by giving **q** a different channel, which would change $\equiv_{\mathcal{N}}$, nor by filtering in the machine after the store has removed the datum. The store’s contract — a candidate that does not match is not consumed — is what makes **q(b,S)** inert in an ungated gate ([4, Lem. 6.5]), and that inertness is what the release lemma rests on.

8.3 Atoms as store operations

Atom	Store call	On commit, emit
$m(a, v)$	<code>produce(a, Loud(v))</code>	—
$q(a, p)$	<code>produce(a, Quiet(p))</code>	—
$d(a, b, c)$	<code>consume([a], [loud], Dup(b, c))</code>	$m(b, v) \mid m(c, v)$
$k(a)$	<code>consume([a], [loud], Kill)</code>	0
$fw(a, b)$	<code>consume([a], [any], Fwd(b))</code>	$m(b, v)$, or $m(b, @p)$ for a quiet datum
$b_l(a, b)$	<code>consume([a], [loud], BindL(b))</code>	$fw(v, b)$
$b_r(a, b)$	<code>consume([a], [loud], BindR(b))</code>	$fw(b, v)$
$s(a, b, c)$	<code>consume([a], [loud], Sync(b, c))</code>	$fw(b, c)$
$e(a)$	<code>consume([a], [loud], Eval)</code>	deploy the normal form of $*v$
$cons(a, b, c)$	<code>consume([a, b], [loud; 2], ConsPar(c))</code>	$m(c, @(p \mid r))$
$cons_m(a, b, c)$	<code>consume([a, b], [loud; 2], ConsMsg(c))</code>	$m(c, @m(u, v))$
$cons_d(a, b, c, f)$	<code>consume([a, b, c], [loud; 3], ConsDup(f))</code>	$m(f, @d(u, v, w))$
$cons_s(a, b, c, f)$	<code>consume([a, b, c], [loud; 3], ConsSync(f))</code>	$m(f, @s(u, v, w))$

Requirement 8.3 (*eval* normalises before it deploys). Under **A** the rule produces $*v$, whose normal form by Requirement 4.4 is the process v quotes. The machine **MUST** construct $*v$ through the smart constructor, which performs the elimination, and **MUST** deploy the result. It **MUST NOT** short-circuit by projecting the payload directly out of the name: the short-circuit is observationally identical here and is the exact step that becomes unsound under **presentation-b**, where $*@p$ and p differ.

Requirement 8.4 (Linear, never persistent). Every **consume** **MUST** pass `persist = false` and an empty `PeekSet`; **install** **MUST NOT** be used. Every rule consumes both participants, and there is no persistent atom in the calculus.

Requirement 8.5 (Deploy). Deploying a target term **MUST** flatten its parallel components in encoding order — which by Requirement 4.4 is a flat multiset of atoms with no descent case — and issue one store call per atom, appending any **CommResult** to the work queue.

Requirement 8.6 (Races are scheduler choices and **MUST** be visible). [4, §3.3] records two deliberate races: two consumers competing for one message, and two producers competing for one consumer, $fw(a, b) \mid m(a, v) \mid q(a, p)$. Under the deterministic profile the store resolves both by order key. The machine **MUST** emit a trace event at every such resolution, naming the alternatives not taken, so the calculus’s nondeterminism is inspectable rather than merely resolved. **RHOBOTS** needs this to draw the choice.

8.4 Name growth

Hazard 8.7 (Names are no longer subterms of the program). [4, §12], open question 3: with the constructors, names grow at run time, and **cons** applied to its own outputs doubles a name’s size per step. [5, §7] says the same thing logically: $Cl_S(T)$ is an infinite namespace with a finite presentation. Nothing the compiler emits does this, but a received name is arbitrary and the machine must survive one.

Requirement 8.8 (Interning and budget). Constructed names **MUST** be hash-consed: a table keyed by content hash, the encoding stored once, channels referring to it by index. The machine **MUST** accept a budget — maximum name size, maximum distinct names — and **MUST** halt with **comb-name-budget** naming the constructor that exceeded it. The default **SHOULD** be generous enough that no compiled source term can reach it, so that hitting it always means a hostile or pathological received name.

Remark 8.9. This is where the implementation repays the papers. [4, §12] asks what name growth costs in practice for the decidability of $\equiv_{\mathcal{N}}$ and for cost accounting, and [5, §7] names it as the bridge to a graded clause. A machine with an interning table and a counter answers it with measurements.

9 The lattice audit, and what the logic needs

[4, §9] organises subsets of the atom set into an expressiveness lattice, a product of a computation axis ($\mathbf{d}$, $\mathbf{e}$) and a name-construction axis ($\mathbf{cons}$ below $\mathbf{cons}_m$, $\mathbf{cons}_d$, $\mathbf{cons}_s$), over a base of $\mathbf{m}$ and the routers. [5, Thm. 6.5] shows occupancy is checkable in linear time, from the atom multiset alone. That multiset is a fact about the compiled term, and the compiler is the only thing that knows it.

Requirement 9.1 (Reporting the point). `f1r3comb-lower` MUST return, alongside the term, the multiset of atom shapes it contains — the $|-|$ of [5, Def. 2.9] — and `f1r3x comb-lattice` MUST print the corresponding lattice point as a pair of coordinates plus the router set. The computation MUST be a count over emitted atoms, and MUST NOT attempt to decide whether a smaller point would suffice for the same source term, which is the open part of [4, §9].

Requirement 9.2 (The multiset is the logic’s input). `f1r3comb-term` MUST expose the atom multiset and the encoding through a stable public API, so that the occupancy checker of [5, §6] can be built against it without depending on `f1r3comb-lower` (Requirement 6.1). It MUST preserve the $\mathbf{m}/\mathbf{q}$ sort distinction in that API, since that distinction is what [5, Rem. 2.5] turns on, and it SHOULD NOT expose anything else: the logic’s characteristic formulae, satisfaction and model checking are [5]’s to build.

Requirement 9.3 (The audit is a regression on the papers). The audit MUST be asserted for a corpus: a source term with no input occupies the constructor-free point; Example 7.13 adds $\mathbf{d}$, $\mathbf{s}$, $\mathbf{e}$, $\mathbf{q}$, $\mathbf{b}_r$; Example 7.14 adds $\mathbf{cons}$ and is the witness the constructor-free fragment cannot express ([4, Prop. 9.5]). If a change moves a corpus term to a different point, the change is either a bug or a result.

Remark 9.4 (A second reason for Decision 2.1). [5, Rem. 2.7] scopes the damage carefully: occupancy, the closure formula and the decidability results hold under either presentation, and only three things are A-only — the name-sort collapse, the drop-free canonical form, and the structural visibility of the $\mathbf{q}/\mathbf{m}$ distinction. The first two have known replacements. The third does not, and it is the one the compiler would otherwise force on the logic.

10 Correctness obligations and testing

Obligation 10.1 (Canonicity). Proved as Obligation 5.8, and tested: for randomly generated K0 terms, and for each a random $\equiv$ -variant obtained by permuting parallel components and re-associating, the compiled encodings MUST be byte-equal.

Obligation 10.2 (The conflation regression). v0.1 stated this as a test that the target congruence omits $*@P = P$. Under Decision 2.1 it does not omit it, and the test changes shape accordingly. A test MUST assert that $Q \mid \mathbf{m}(@Q, @P)$ is stuck for a nontrivial Q — that the machine does not implement [4, Prop. 3.5] — and a second MUST assert Requirement 4.2: no process is equal to an $\mathbf{e}$ atom unless it was written as one. The first is the symptom, the second the cause.

Obligation 10.3 (Operational correspondence). [4, Lem. 7.7]: for a source communication, $\llbracket \text{for}(y \leftarrow x)P \rrbracket_s \mid \mathbf{m}(\llbracket x \rrbracket, @\llbracket R \rrbracket) \longrightarrow^* T$ with $T \approx \llbracket P\{ @R/y \} \rrbracket_s$ in $k+3$ steps followed by at most

k administrative steps. The machine **MUST** be tested against both halves: step counts exactly, for a corpus with known k ; and the residue by differential testing. Note that [4, Lem. 7.7]’s (o4) case now holds under either presentation — under A both sides deliver $*@[[R]]$, which the equation makes equal to $[[R]]$.

Obligation 10.4 (Differential testing against B3LL0WS). For a corpus of K0 programs, run the source under B3LL0WS and the compiled term under **f1r3comb-run**, both deterministic, to quiescence, and compare observations. The observation **MUST** be the multiset of messages at names that are encodings of source names — the encoded observer class of [4, Thm. 7.9] — and **MUST NOT** include the seed-derived apparatus, which by [4, Lem. 7.6] no encoded context can name. A mismatch **MUST** report the first divergent step of both traces.

Obligation 10.5 (A against B). The corpus **MUST** be run under both presentations and the observations compared. [4, Def. 3.3] asserts that no rule, lemma or proof distinguishes them; this is that assertion made executable, and it is the check that the **presentation-b** seam has not rotted. The encodings will differ and **MUST** be expected to; only the observations are compared.

Obligation 10.6 (Golden encodings). Every example in this specification and in [4, §6] **MUST** be a golden file with source, term, encoding and hash, per presentation, and the goldens **MUST** be regenerated only by an explicit command.

Obligation 10.7 (Property tests). A generator for K0 terms **MUST** exist, and on its output: lowering is total; the term contains no target variable (Requirement 4.16) and, under A, no drop (Requirement 4.4); no constructor has a repeated subject or a delivery channel among its inputs (Requirement 7.10); every seed power is outside the constructor image (Requirement 5.5); the size bound of Requirement 7.16 holds; deploy-then-quiesce terminates for terminating sources.

Obligation 10.8 (Seed independence, as a property). [4, Lem. 7.8] says two seeds give bisimilar terms. The compiler never uses two seeds for one name, but the property is worth testing as a check on the machine: compile with the term-determined seed and with an injected alternative, run both, compare observations. A failure here is a machine bug, not a compiler bug.

11 Errors and diagnostics

Requirement 11.1 (Diagnostic model). Diagnostics **MUST** follow [6, §9]: a stable code, a span, a message, no panic on any input, and every independent error reported in one pass.

Code	Phase	Meaning
comb-level	1	A form above K0; names the level that would accept it
comb-unguarded-drop	2	A drop of a name not bound by an enclosing input; warning under A, error under B
comb-drop-quote	2	A literal $*@Q$; collapsed by the equation under A, unsound under B
comb-open	4	A free variable or free name survived grounding
comb-unforgeable	4	An unforgeable name; no target former
comb-occ-unclassified	5	Internal: an occurrence matching no kind
comb-presentation-mismatch	deploy	Artefact compiled under the other presentation (Requirement 2.3)
comb-feature-mismatch	deploy	Artefact compiled with a different shortcuts setting
comb-name-budget	run	Name size or count budget exceeded
comb-decode	any	A source tag, or the drop tag in an A build, where a target tag was expected (Requirements 4.13, 4.14)

12 Performance, portability, dependencies

Requirement 12.1 (Complexity). Lowering **MUST** be $O(n \log n)$ in the size of the normal form, the log from the memo table of Requirement 7.8, plus the chain term of Requirement 7.16. Encoding **MUST** be linear and computed once per node. A machine step **MUST** be $O(1)$ expected in the size of the store. The atom multiset of Requirement 9.1 **MUST** be linear, matching [5, Thm. 6.5].

Requirement 12.2 (Targets). On the reference machine of [6, §11]: compile a 10^4 -node source term in under 10 ms; deploy 10^5 atoms in under 100 ms; sustain 10^6 machine steps per second free-running. Budget figures for a first implementation, to be re-stated once measured.

Requirement 12.3 (Portability). Every crate **MUST** build for `wasm32-unknown-unknown` without `wasm-bindgen`, **MUST** carry `forbid(unsafe_code)`, and **MUST NOT** take an external dependency. Hashing comes from `kindling-norm`.

Requirement 12.4 (Determinism). Given the same source, presentation and configuration, the compiled term, its encoding, its hash, the event stream and the final scene **MUST** be bit-identical across runs, hosts, core counts and WebAssembly versus native.

13 What the implementation asks of the papers

Two items from v0.1 are discharged by draft 3’s revision and are recorded as closed: the binder rules now produce forwarders and the arity slips in `fw` and in the constructors’ delivery channel are fixed ([4, Rem. 3.4]); and the presentation question, which v0.1 met only obliquely, is now posed explicitly. The rest stand.

1. **The implementation argument for B does not hold** (Section 2.3). [4, §4] reads the F1R3FLY normaliser as evidence for B. Sorting a parallel composition is the monoid laws, common to both presentations; de Bruijn indexing applies to source binders and the combinators have none; and the one presentation-specific rewrite the code performs, $*@P \rightarrow P$, is A’s equation. Proposed amendment: keep §4’s structural argument, which is sound, and withdraw the appeal to the implementation, or restate it as what a future optimising decoder would need.
2. **Seed powers are constructible** (Erratum 5.4). $s^{w0} = @m(s^w, @0)$ is in the image of cons_m . Proposed amendment: $s^{w0} = @b_l(s^w, s^w)$. [4, Lem. 6.2] is unchanged; the freshness it gives becomes freshness against reachable names, which is the property the constructors make relevant.
3. **The seed of a quotation must be term-determined** (Decision 5.6). [4, Def. 6.6]’s “ w is the position of that quotation in the enclosing term” makes one source name compile to several target names. Proposed amendment: $[@P] := @[P]_{@P}$, which [4, Lem. 6.2] already licenses.
4. **The enumeration order of occurrences is observable** (Requirement 7.3). Proposed amendment: fix it, or state that the translation is defined only up to the induced bijection on proxies.
5. **Coincident occurrence kinds** (Requirement 7.4). [4, Table 1]’s note is about occurrences, not kinds. Proposed amendment: say so, and say whether fusing two occurrences into one proxy is permitted.

6. **The shortcuts change the atom set** (Requirement 7.12). [4, Rem. 3.10] says it; [4, Rem. 6.7] offers the shortcuts without repeating it. Proposed amendment: cross-reference them.
7. **The formation table's m row is presentation-specific.** [5, Rem. 2.6] already raises this and this specification seconds it from the implementation side: [4, §3.2] presents formation as common to both presentations while sorting the message payload as a name, which is A's sorting, and §3.3 then re-sorts it. Proposed amendment: split the row, as [5, Def. 2.3] does.
8. **The source calculus of [4, §2] is not the source calculus of [6, Def. 2.6]** (Obligation 3.3). The first omits $*@P \equiv P$ and uses semantic substitution; the second asserts it and $\text{Norm}::\text{eval}$ applies it. They agree on guarded-drop terms, which is why the mismatch has gone unremarked. Proposed amendment: state the agreement as a lemma in §2, since with Decision 2.1 the target now satisfies the equation and the source presentation is the last place it does not.

14 Open questions and deferred work

1. **Name growth, measured** (Hazard 8.7). The machine is the instrument [4, §12] question 3 and [5, §7] both want.
2. **The quiet constructors** ([4, Rem. 7.12]). Should every constructor have a quiet variant emitting $\mathbf{q}(c, \cdot)$? Under Decision 4.9 the loud and quiet cases are already two arms of one enum, so a **quiet-constructors** feature is cheap to try, and Obligation 10.4 says whether an encoded observer can tell.
3. **Receptivity.** [5, open item 5]. If the logic gains $\langle\langle\phi\rangle\rangle\psi$, the $\mathbf{m}/\mathbf{q}$ argument for A weakens and the presentation question is worth reopening. Until then it is the decisive argument.
4. **An optimising decoder.** Remark 2.2: the one development that would force B. If a backend that discharges administrative forwarders at decode time is wanted, it is a **presentation-b** backend, and Obligation 10.5 is what keeps that path open.
5. **The constructors-only presentation** ([4, §10]). Deferred until observation is defined over the constructor family.
6. **Composition with the π compiler.** [8] encodes π into ρ ; composing gives π into the combinators. The composite's canonicity is a new obligation, not an inherited one.
7. **A decompiler.** A term in the image of $\llbracket - \rrbracket$ should be recoverable, and a round-trip test would be a strong canonicity check; whether the image is decidable is open.

15 Delivery

M	Deliverable	Done when
M0	<code>f1r3comb-term</code>	Sorts, atom enum, drop elimination, encoding, hash, printer, decoder, atom multiset; tag-disjointness and rigidity tests; round-trip property test
M1	<code>f1r3comb-lower</code> , no (o5)	Level, guard and closedness checks; seeds; kinds (o1)–(o4); Example 7.13 golden; canonicity property test
M2	Constructor chains	Kind (o5), Requirements 7.9–7.11; Example 7.14 golden; lattice audit and the public API of Requirement 9.2
M3	<code>f1r3comb-space</code> , <code>f1r3comb-run</code>	Matcher, rule table, deploy, scheduler, trace; conflation regression; step-count tests
M4	<code>f1r3comb-check</code>	Differential testing against B3LL0WS; the A/B comparison; corpus; size and complexity tests
M5	F1R3X subcommands, WASM	<code>comb</code> , <code>comb-hash</code> , <code>comb-run</code> , <code>comb-lattice</code> ; <code>wasm32</code> build; performance measured and Requirement 12.2 re-stated

M0 through M2 are the compiler and are independently useful: they give [5] a term type and an atom multiset, and they give draft 3 a machine-checked version of its own examples. The `presentation-b` seam is built at M0 and exercised at M4; it is not a separate milestone, and it will rot if Obligation 10.5 is not run.

16 Changes from v0.1

v0.1	v0.2
No presentation question	Section 2: A chosen, B behind presentation-b , with the [4, §4] argument answered
Hazard: <code>Norm::eval</code> applies a fatal equation	Not a hazard under A; one rewrite, sound on both sides. The hazard returns under B (Requirement 4.5)
Requirement: target congruence omits $*@P = P$	Requirement 4.1: the drop and the guard are distinct constructors. The equation is optional; the conflation is the defect
Guarded-drop check is a gating phase	A diagnostic under A, gating under B (Requirement 3.4), with Obligation 3.3 explaining why
Decision: <code>q</code> carries a name	Withdrawn. <code>q</code> carries a process and the sort asymmetry is load-bearing (Decision 4.9, Remark 4.10)
Datum: name plus a <code>quiet</code> flag	A two-case enum over two sorts
Tag table: no drop, <code>cons</code>	Drop tag reserved and rejected under A; <code>cons</code> per [4, §3.2]; <code>e</code> given its own tag distinct from the drop
Lattice audit only	Plus Requirement 9.2: the multiset and the sort distinction are a public API for [5]
Six items asked of the paper	Eight, of which two from v0.1 are discharged by draft 3's revision and three are new
Cross-references to draft 3	Renumbered throughout against the revision; [5] added

References

- [1] C.B. Wells, M. Stay and L.G. Meredith. *Behavior in higher-order languages: admissible observers, relative pushouts, and the encoding of extended theories*. In preparation, F1R3FLY.io, 2026.
- [2] F1R3FLY.io. *Bytecode or normal form: what the executive should execute*. publications/campf1r3/bytecode-or-normal-form, 2026.
- [3] F1R3FLY.io. *CampF1R3: an in-memory tuple space*. publications/campf1r3/campf1r3-spec, 2026.
- [4] L.G. Meredith and M. Stay. *Name-free combinators for the rho calculus: a repair, a linear encoding, and the one operation that was missing*. Draft 3, publications/rho-combinators, September 2026.
- [5] F1R3FLY.io. *A spatial-behavioural logic for the rho combinators: structural formulae, one context modality, and decidable occupancy of the expressiveness lattice*. Draft 2, publications/rho-combinators, September 2026.
- [6] F1R3FLY.io. *K1ndllng: a parser and a normaliser for the kernel of rholang served by CampF1R3, as composable crates*. Specification v0.2, publications/campf1r3/k1ndllng-spec, 17 September 2026.

- [7] L. G. Meredith and M. Radestock. *A reflective higher-order calculus*. ENTCS 141(5):49–67, 2005.
- [8] L. G. Meredith. *Namespaces without a name server: freshness by communication, and a fully abstract encoding of the π -calculus in the rho calculus*. `publications/pi2rhobugfix`, F1R3FLY.io, 2026.
- [9] L. G. Meredith and M. Stay. *Name-free combinators for concurrency*. Working draft, `rho4u`.
- [10] N. Yoshida. *Minimality and separation results on asynchronous mobile processes: representability theorems by concurrent combinators*. Theoretical Computer Science 274(1–2):231–276, 2002.